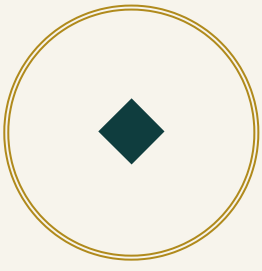
COMPLEX ENGINEERING PROBLEM · SEMESTER PROJECT

DegreeChain: Blockchain-Based Degree Verification System

A tamper-proof digital record of university degrees, verifiable by anyone in seconds

Solidity smart contract | Ethereum Sepolia | React + ethers.js | SHA-256 | No cryptocurrency

Course	Blockchain & Technology Applications (BTA)
Submitted by	_____
Roll number(s)	_____
Submitted to	_____
Department	_____
Date	November 2026
Live application	`https://degreechain.vercel.app`
Source code	`github.com/shahid786-dev/Blockchain-Based-Degree-Verification`



Abstract

Fake and altered academic degrees are a long-standing problem, and checking whether a real degree is genuine is slow because it depends on contacting the issuing university or the regulator. This project presents **DegreeChain**, a degree verification system that uses a public blockchain purely as a **tamper-proof digital record**; no cryptocurrency, token or payment logic is involved. Universities approved by a regulator record each degree in a Solidity smart contract together with the SHA-256 fingerprint of the degree document. The record cannot be edited afterwards, only revoked with a stated reason. Any employer or institution can then verify a degree for free, in seconds, by entering its certificate ID, scanning its QR code, or uploading the degree file: if even one character of the file has been changed, its fingerprint no longer matches and the system reports it as not verified. The contract is deployed on the Ethereum Sepolia test network and the React web application is live on Vercel. All functional and security scenarios that were executed passed.

Contents

1. Introduction

2. Problem Statement and CEP Attributes

3. Objectives and Scope

4. Background and Related Work

5. Proposed Solution: Why Blockchain?

6. System Design

6.1 Architecture · 6.2 Actors and use cases · 6.3 Roles and permissions · 6.4 Data model · 6.5 Certificate lifecycle

7. Workflows

7.1 Issuing a degree · 7.2 Verifying a degree

8. Implementation

8.1 Technology stack · 8.2 Smart contract · 8.3 Document hashing · 8.4 Frontend

9. Deployment

10. Application Walkthrough (Screenshots)

11. Testing and Results

12. Security, Privacy and Cost Analysis

13. Limitations and Future Work

14. Conclusion

References

Appendix A: Smart Contract API · Appendix B: Running the Project

1. Introduction

A university degree is one of the most important documents in a person's career. Employers, licensing bodies, embassies and other universities all rely on it, yet the way it is checked has hardly changed: the document is a printed sheet or a PDF, and the only reliable way to confirm it is to ask the institution that issued it. In Pakistan this usually means going through the Higher Education Commission (HEC) attestation process or writing to the university's examination department, which takes days to weeks.

Because checking is slow, it is often skipped, and that gap is exploited. Degree certificates can be produced with ordinary design software, and a genuine certificate can be edited to change a grade, a year or a name. Pakistan has also been at the centre of an international fake-degree scandal (the Axact case reported by *The New York Times* in 2015), which shows the scale the problem can reach.

Blockchain technology offers a different model. A blockchain is a shared ledger in which records, once written, cannot be changed or deleted without the change being visible to everyone. If universities write a fingerprint of each degree to such a ledger at the time of issue, anyone can later compare a document against that record without contacting the university. This report describes the design, implementation, deployment and testing of such a system.

> **Scope note.** As required by the project brief, the blockchain is used only as a digital record store. The system has no token and no cryptocurrency features. It runs on the Sepolia *test* network, where the small amount of "test ETH" needed to pay transaction fees is free and has no monetary value.

2. Problem Statement and CEP Attributes

2.1 Problem statement

Design and implement a system that allows universities to issue academic degrees as tamper-proof digital records, and allows any third party to verify the authenticity and current validity of a degree quickly and independently, without relying on a central authority being available and without storing the degree documents themselves publicly.

2.2 Issues with the current process

Issue	Effect
Paper and PDF documents are easy to forge or edit	A fake or altered degree can look identical to a real one.
Verification depends on the issuing institution	Employers must email or visit the university or regulator and wait for a reply.
Slow turnaround (days to weeks)	Verification is often skipped under hiring pressure.
Central databases can be altered by insiders	A single administrator can change a record without anyone noticing.
No standard way to mark a degree as cancelled	A degree withdrawn for misconduct can still be presented as valid.

2.3 Complex Engineering Problem attributes

The problem satisfies the attributes of a Complex Engineering Problem as follows.

Attribute	How it applies to this project
WP1 Depth of knowledge	Requires knowledge of distributed ledgers, smart contracts (Solidity, EVM), cryptographic hashing, public-key wallets, and web development.
WP2 Conflicting requirements	Transparency vs. student privacy; immutability vs. the need to correct mistakes; public verification vs. restricting who may issue; low cost vs. on-chain storage.
WP3 Depth of analysis	Choosing what to store on-chain (details vs. hash), designing the role model, handling revocation without deletion, and measuring gas cost.
WP4 Familiarity of issues	Blockchain-based credentialing is a relatively new field with few established local practices.
WP5 Applicable codes and standards	SHA-256 (FIPS 180-4), Ethereum JSON-RPC and EIP-1193 wallet interface, Solidity best practices, data-privacy considerations.
WP6 Stakeholder involvement	Regulator (HEC), universities, students, employers and other verifiers, each with different needs and permissions.

Attribute	How it applies to this project
WP7 Interdependence	The smart contract, wallet, RPC node, hashing, frontend and hosting must all work together; a change in the contract affects the ABI, bytecode and UI.

3. Objectives and Scope

3.1 Objectives

1. Let a regulator approve, suspend and re-activate the universities that are allowed to issue degrees.
2. Let an approved university record a degree on the blockchain together with the SHA-256 fingerprint of the degree document.
3. Give every degree a permanent certificate ID and a QR code that links to its verification page.
4. Let anyone verify a degree for free, with no account or wallet, by certificate ID, QR code or by uploading the file.
5. Detect any modification of a degree document, down to a single character.
6. Allow the issuing university to revoke a degree with a recorded reason, without deleting history.
7. Keep the degree file private: it must never be uploaded anywhere.
8. Deploy the system publicly so it can be demonstrated live.

3.2 Scope

In scope: regulator, university and verifier roles; issue, verify and revoke; QR codes; document fingerprinting; deployment to a public test network and a public website.

Out of scope: cryptocurrency or tokens; storing the degree files themselves; bulk import of existing records; integration with real university or HEC systems; production (mainnet) deployment.

4. Background and Related Work

4.1 Blockchain

A blockchain is an append-only ledger replicated across many computers (nodes). Transactions are grouped into blocks, and each block contains the cryptographic hash of the previous one, so changing an old record would change every hash after it and be rejected by the other nodes. Bitcoin introduced this design in 2008 for payments; Ethereum (2015) extended it with **smart contracts**, programs stored on the chain whose code and data are public and whose rules are enforced by every node.

4.2 Smart contracts and the EVM

Ethereum smart contracts are usually written in Solidity and compiled to bytecode for the Ethereum Virtual Machine (EVM). Functions that *change* state are sent as transactions signed by a wallet and cost a fee called *gas*; functions that only *read* state (view functions) are free and need no wallet. DegreeChain relies on exactly this split: issuing is a signed transaction, verifying is a free read.

4.3 Cryptographic hashing

SHA-256 maps any input to a 256-bit (32-byte) value. It is practically impossible to find two different files with the same hash, and changing a single bit of the input changes about half of the output bits. Storing the hash of a degree PDF therefore proves which exact file was issued, without revealing the file.

4.4 Related systems

System	Approach	Difference from DegreeChain
HEC degree attestation (Pakistan)	Online application, document check by HEC staff, physical or digital stamp.	Centralised and manual; verification takes days.
University of Nicosia (2014)	Among the first universities to anchor certificates on the Bitcoin blockchain.	Single institution; no regulator or approval model.
Blockcerts (MIT Media Lab, 2016)	Open standard for blockchain-anchored credentials with a verifier tool.	General standard; DegreeChain adds an on-chain regulator role, revocation reasons and file-upload verification.
Central database portal	University publishes a lookup page backed by its own database.	Records can be edited by insiders; depends on the university's server being online.

5. Proposed Solution: Why Blockchain?

DegreeChain moves the *record of issue* from the university's private filing system to a public ledger, while keeping the document itself with the student. The core idea is simple: **at the moment of issue, the university writes the degree details and the fingerprint of the degree file to a smart contract.** From then on, the contract is the single source of truth that anyone can query.

Requirement	Central database	DegreeChain (blockchain)
Records cannot be altered	Admin can edit rows silently	Contract has no edit function; history is permanent
Forged file detected	Only if staff compare manually	Automatic SHA-256 comparison
Verification without contacting university	Only if a portal exists and is online	Free public read from any Ethereum node
Who may issue is enforced	Application logic, can be bypassed by DB access	Enforced by contract code on every node
Revocation is auditable	Row can be deleted	Revocation is a recorded event with a reason
Single point of failure	Yes (one server)	No (thousands of nodes)

5.1 Key design decisions

- **Hash on-chain, file off-chain.** Only the 32-byte SHA-256 hash of the document is stored. The file is hashed in the user's browser and never uploaded.
- **Permissioned issuing on a public chain.** Anyone can read, but only wallets approved by the regulator can issue.
- **No edit, only revoke.** A mistake is handled by revoking (with a reason) and issuing a new certificate, so the audit trail is never lost.
- **One document, one certificate.** The contract refuses to register the same document hash twice.
- **Suspension does not invalidate history.** A suspended university cannot issue new degrees, but degrees it issued earlier remain valid.

6. System Design

6.1 Architecture

The system has three layers: users with their browsers and wallets, a static React web application hosted on Vercel, and the CertificateRegistry smart contract on the Ethereum Sepolia network. There is no backend server and no database: all state lives in the contract. Read operations go through a public Sepolia RPC endpoint; write operations are signed by the user's MetaMask wallet.

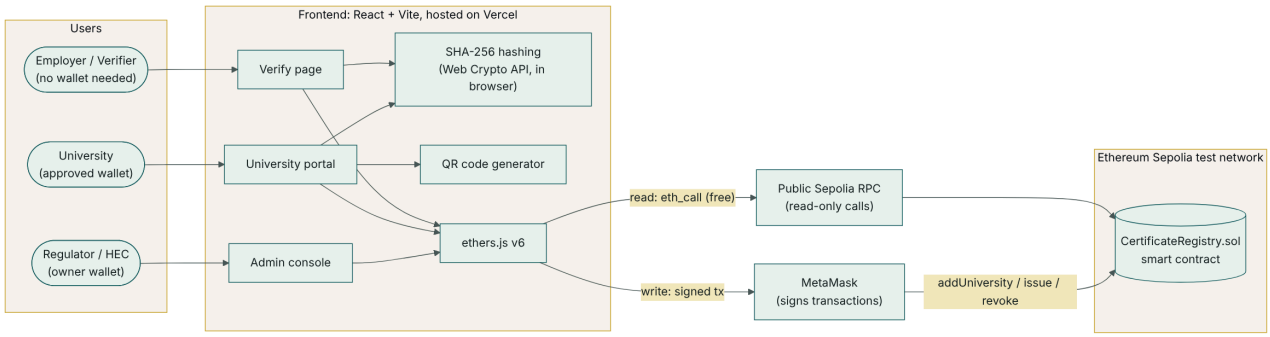


Figure 1. System architecture

6.2 Actors and use cases

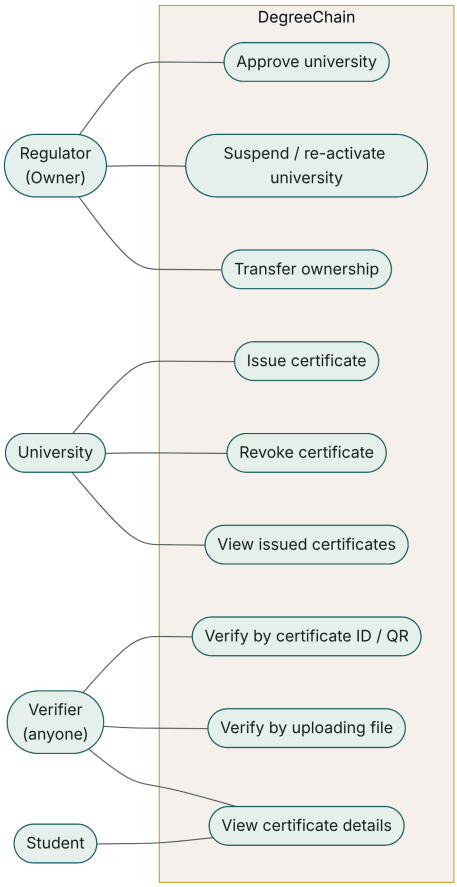


Figure 2. Use case diagram

Actor	Description
Regulator (Owner)	The wallet that deployed the contract, representing a body such as HEC. Approves and suspends universities and can hand ownership to a new wallet.
University	An approved wallet. Issues degrees, lists the degrees it issued, and revokes its own degrees.
Student	Receives the certificate ID and QR code and shares them, for example on a CV.
Verifier	Anyone (employer, embassy, university admissions). Verifies without any account or wallet.

6.3 Roles and permissions

Action	Regulator	Active university	Suspended university	Anyone
Approve / suspend a university	✔	✘	✘	✘
Transfer ownership	✔	✘	✘	✘
Issue a certificate	only if also approved	✔	✘	✘
Revoke a certificate	only the university that issued it			✘
Verify / view a certificate	✔	✔	✔	✔ (free)

6.4 Data model

The contract stores two record types, `University` and `Certificate`, and several mappings that allow fast lookups by certificate ID, by document hash and by issuing university. The certificate ID

is computed as `keccak256(issuer, rollNumber, documentHash, counter, chainId)`, which makes it unique and unpredictable.

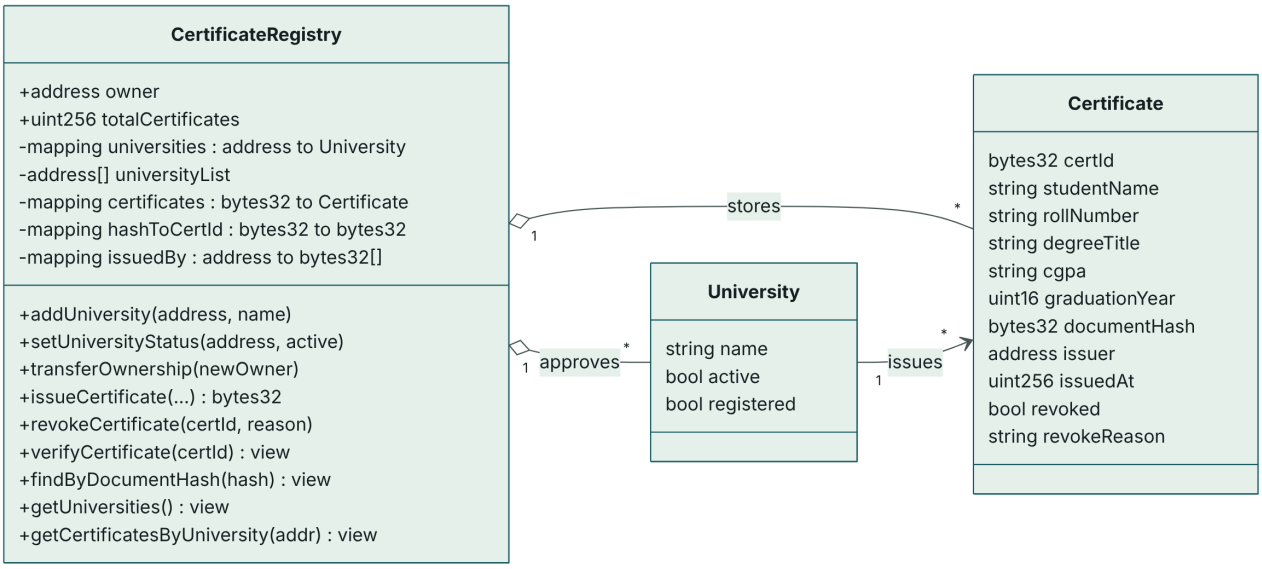


Figure 3. Smart contract data model

6.5 Certificate lifecycle

A certificate has only two states. It becomes *Valid* when issued and can move to *Revoked* once, only by its issuer. There is no transition back and no way to edit the details.

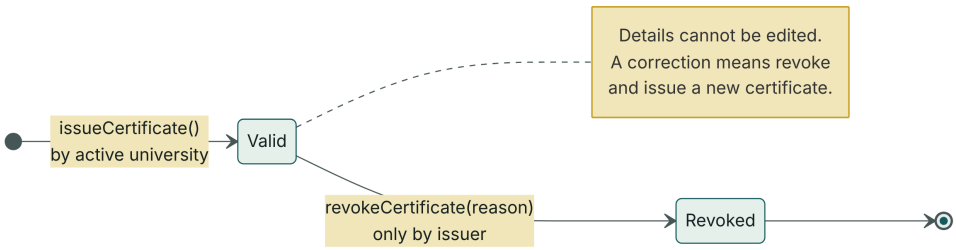


Figure 4. Certificate state diagram

7. Workflows

7.1 Issuing a degree

The university staff member fills in the student details and attaches the degree PDF. The browser computes the SHA-256 hash locally and sends the details and hash to the contract through MetaMask. The contract checks that the sender is an active university, that no field is empty and that the hash has not been used before, then stores the record and emits a `CertificateIssued` event containing the new certificate ID. The portal reads the stored record back and shows it with a QR code.

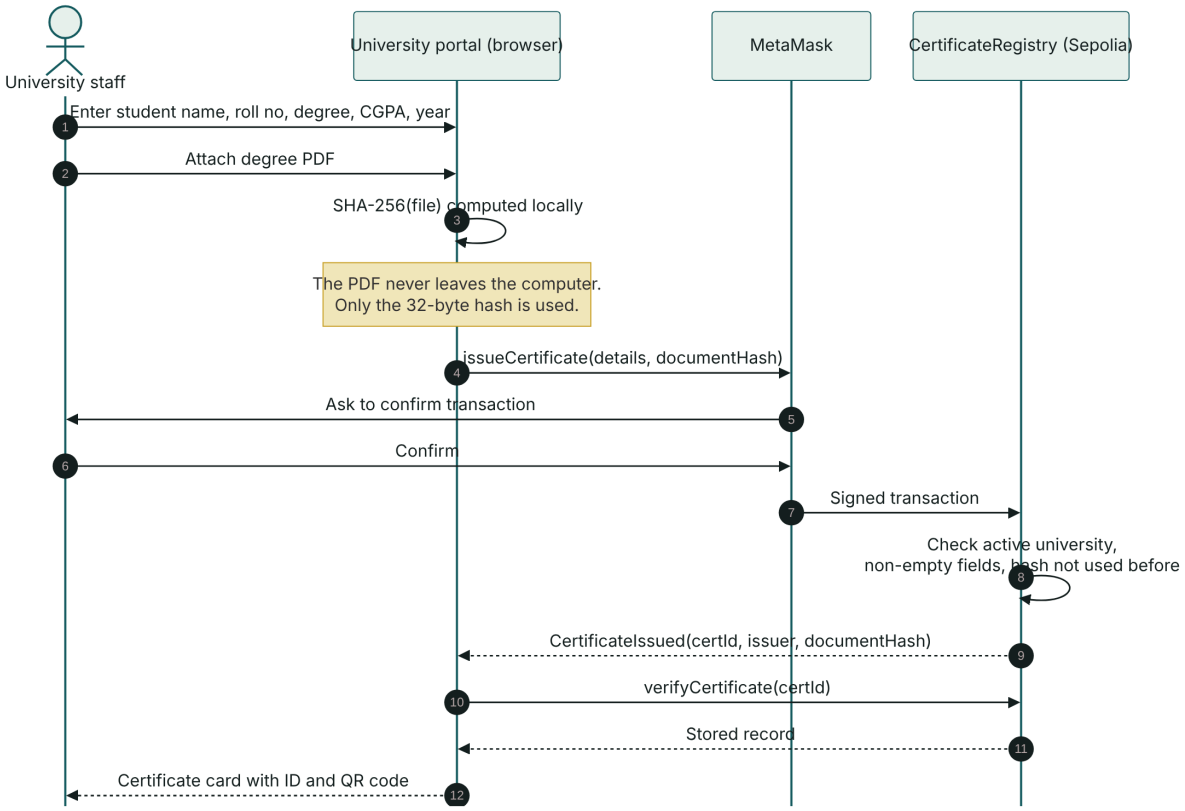


Figure 5. Sequence diagram for issuing a degree

7.2 Verifying a degree

A verifier can start from a certificate ID (typed or scanned from the QR code) or from the document itself. With a file, the browser computes its hash and asks the contract which certificate, if any, was registered with that hash. The result is one of four outcomes: **Genuine and valid**, **Revoked** (with reason), **Not verified** because the ID does not exist, or **Not verified** because the file's fingerprint is not on record. After an ID lookup the verifier can also drop the candidate's PDF to confirm it is exactly the registered file.

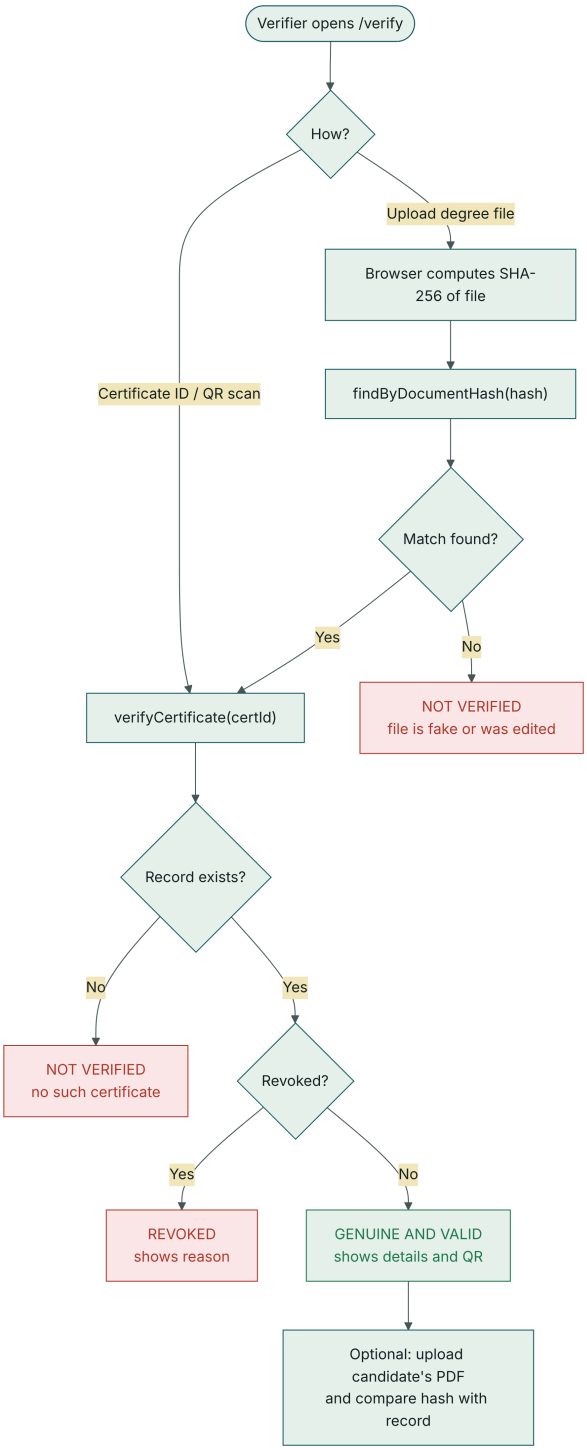


Figure 6. Verification flowchart

8. Implementation

8.1 Technology stack

Layer	Technology	Purpose
Smart contract	Solidity 0.8.24, optimizer 200 runs	On-chain registry, roles, rules
Blockchain	Ethereum Sepolia test network (chain ID 11155111)	Public, immutable record store
Development and tests	Hardhat, Chai, Ethers v6; Foundry (Anvil) for end-to-end runs	Compile, unit test, local chain
Frontend	React 18, Vite 5, React Router 6	Single-page web application
Blockchain client	ethers.js v6	Contract calls and transactions
Wallet	MetaMask (EIP-1193)	Signs regulator and university actions
Hashing	Web Crypto API (SHA-256)	Document fingerprint in the browser
QR codes	qrcode.react	Link to the public verify page
Hosting and CI	GitHub + Vercel (auto-deploy on push)	Public website

8.2 Smart contract

The `CertificateRegistry` contract (239 lines) contains all business rules. Access is controlled with two modifiers, `onlyOwner` and `onlyActiveUniversity`, and every failure raises a named custom error (for example `NotIssuer` or `DocumentAlreadyRegistered`) that the frontend turns into a readable message. The issuing function is shown below.

```
function issueCertificate(
    string memory studentName, string memory rollNumber,
    string memory degreeTitle, string memory cgpa,
    uint16 graduationYear, bytes32 documentHash
) external onlyActiveUniversity returns (bytes32 certId) {
    if (bytes(studentName).length == 0 || bytes(rollNumber).length == 0 ||
        bytes(degreeTitle).length == 0 || bytes(cgpa).length == 0 ||
        graduationYear == 0) revert EmptyField();
    if (documentHash == bytes32(0)) revert InvalidDocumentHash();

    bytes32 existing = hashToCertId[documentHash];
    if (existing != bytes32(0)) revert DocumentAlreadyRegistered(existing);

    certId = keccak256(abi.encode(msg.sender, rollNumber, documentHash,
                                  totalCertificates, block.chainid));
    Certificate storage c = certificates[certId];
    c.certId = certId;  c.studentName = studentName;  c.rollNumber = rollNumber;
    c.degreeTitle = degreeTitle;  c.cgpa = cgpa;  c.graduationYear = graduationYear;
    c.documentHash = documentHash;  c.issuer = msg.sender;  c.issuedAt = block.timestamp;

    hashToCertId[documentHash] = certId;
    issuedBy[msg.sender].push(certId);
    totalCertificates += 1;
    emit CertificateIssued(certId, msg.sender, documentHash, rollNumber);
}
```

Revocation checks that the caller is the original issuer and that the certificate is not already revoked, then sets the `revoked` flag and stores the reason. Verification is a `view` function, so it is free:

```
function verifyCertificate(bytes32 certId) external view
    returns (bool exists, bool valid, Certificate memory cert, string memory universityName)
{
    cert = certificates[certId];
    exists = cert.issuer != address(0);
    valid = exists && !cert.revoked;
    universityName = universities[cert.issuer].name;
}
```

8.3 Document hashing in the browser

The file chosen by the user is read into memory and hashed with the browser's built-in Web Crypto API. Nothing is sent over the network except the resulting 32-byte value.

```
export async function sha256File(file) {
  const buffer = await file.arrayBuffer();
  const digest = await crypto.subtle.digest("SHA-256", buffer);
  return "0x" + Array.from(new Uint8Array(digest))
    .map(b => b.toString(16).padStart(2, "0")).join("");
}
```

8.4 Frontend application

Page	Wallet needed	Function
Home	No	Explains the problem, how the system works and why blockchain is used.
Verify	No	Verify by certificate ID / QR link (`/verify/<id>`) or by uploading the file; optional file cross-check after an ID lookup.
University	Approved university	Issue form with file drop and live hash; shows the issued certificate with QR; searchable list of issued degrees with revoke.
Admin	Owner	Statistics, approve university, suspend and re-activate, network and contract information.
Deploy	Any (becomes owner)	One-time page that deploys the contract from the browser using MetaMask, so no private key is ever typed into a terminal.

The application detects the connected wallet's role by calling `owner()` and `getUniversity(address)`, shows the role in the navigation bar, and automatically asks MetaMask to switch to Sepolia. Contract errors such as `NotActiveUniversity` or insufficient test ETH are translated into plain-language messages.

9. Deployment

The smart contract was deployed to Sepolia from the application's own `/deploy` page using a MetaMask account created only for this project. The address of the deployed contract is stored in the repository (`frontend/src/contract/deployments/sepolia.json`), and Vercel rebuilds the website automatically on every push to the `main` branch.

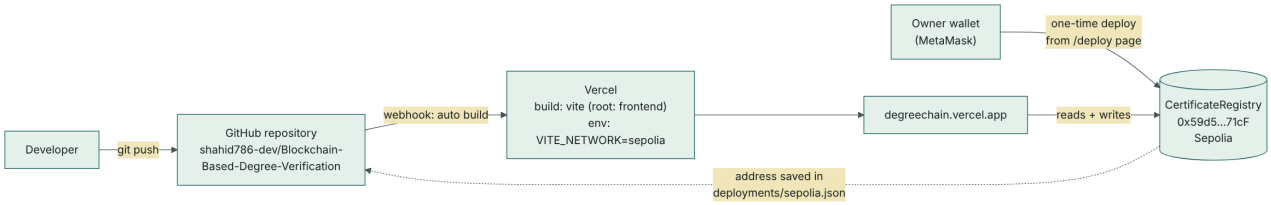


Figure 7. Deployment pipeline

Live website	`https://degreechain.vercel.app`
Network	Ethereum Sepolia test network (chain ID 11155111)
Contract address	`0x59d52b03ff5bFB75b007F8E55F0d621b3a5F71cF`
Owner (regulator) wallet	`0x262B481a5f847E7c73536c530bA7538F5639D2ac`
Deployment transaction	`0xa25d579c779c52bd7695a5b4fb619369cd219f08f104bb1cb6338833a2ee624b`
Deployed on	5 November 2026
Compiler	solc 0.8.24+commit.e11b9ed9, optimizer enabled (200 runs), EVM version paris
Block explorer	`sepolia.etherscan.io/address/0x59d52b03ff5bFB75b007F8E55F0d621b3a5F71cF`
Source repository	`github.com/shahid786-dev/Blockchain-Based-Degree-Verification`

10. Application Walkthrough

The following screenshots were taken from the production build of the web application while running the complete flow. All student names and universities are fictional sample data.

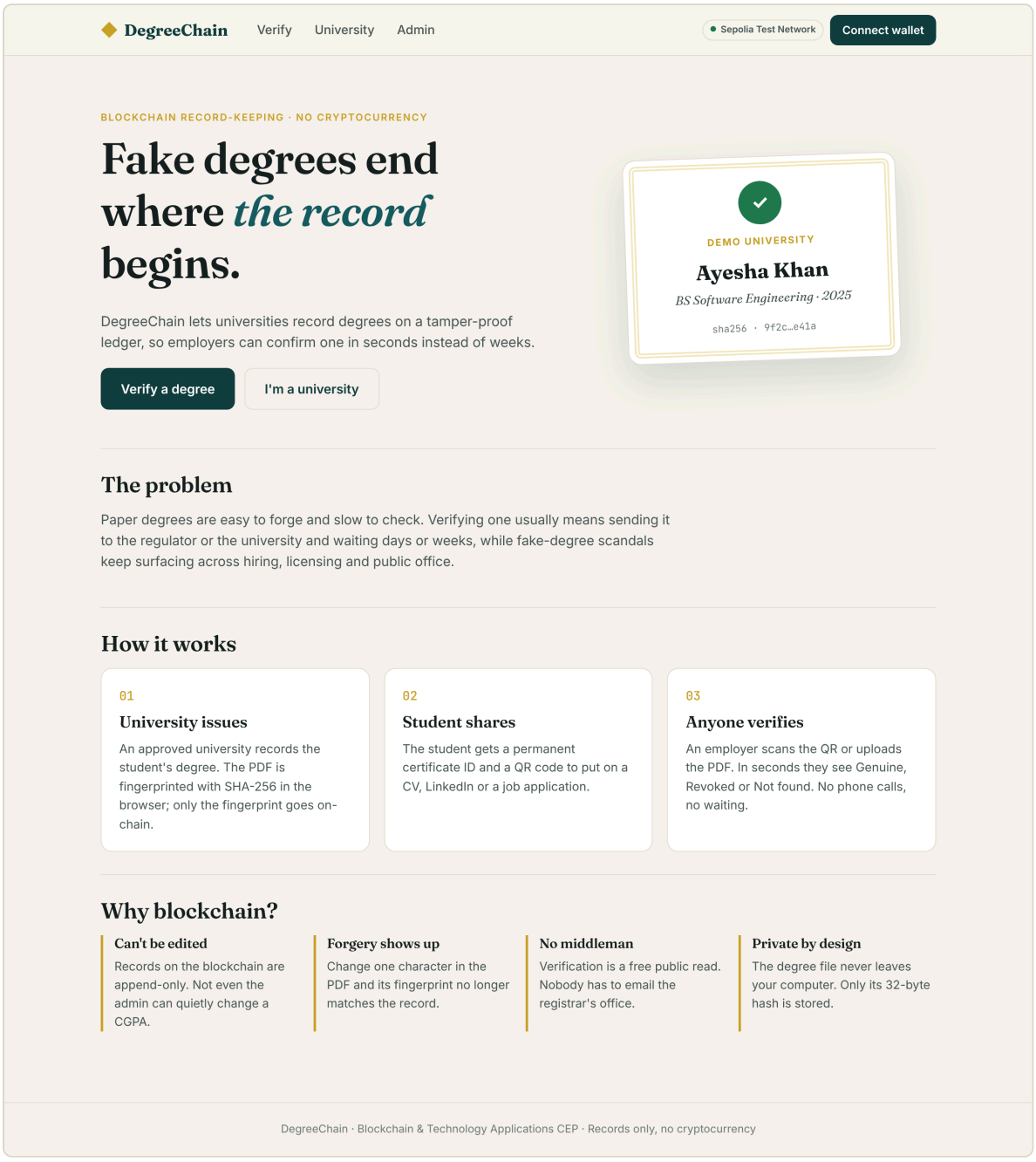


Figure 8. Home page explaining the problem, the process and the benefits of blockchain

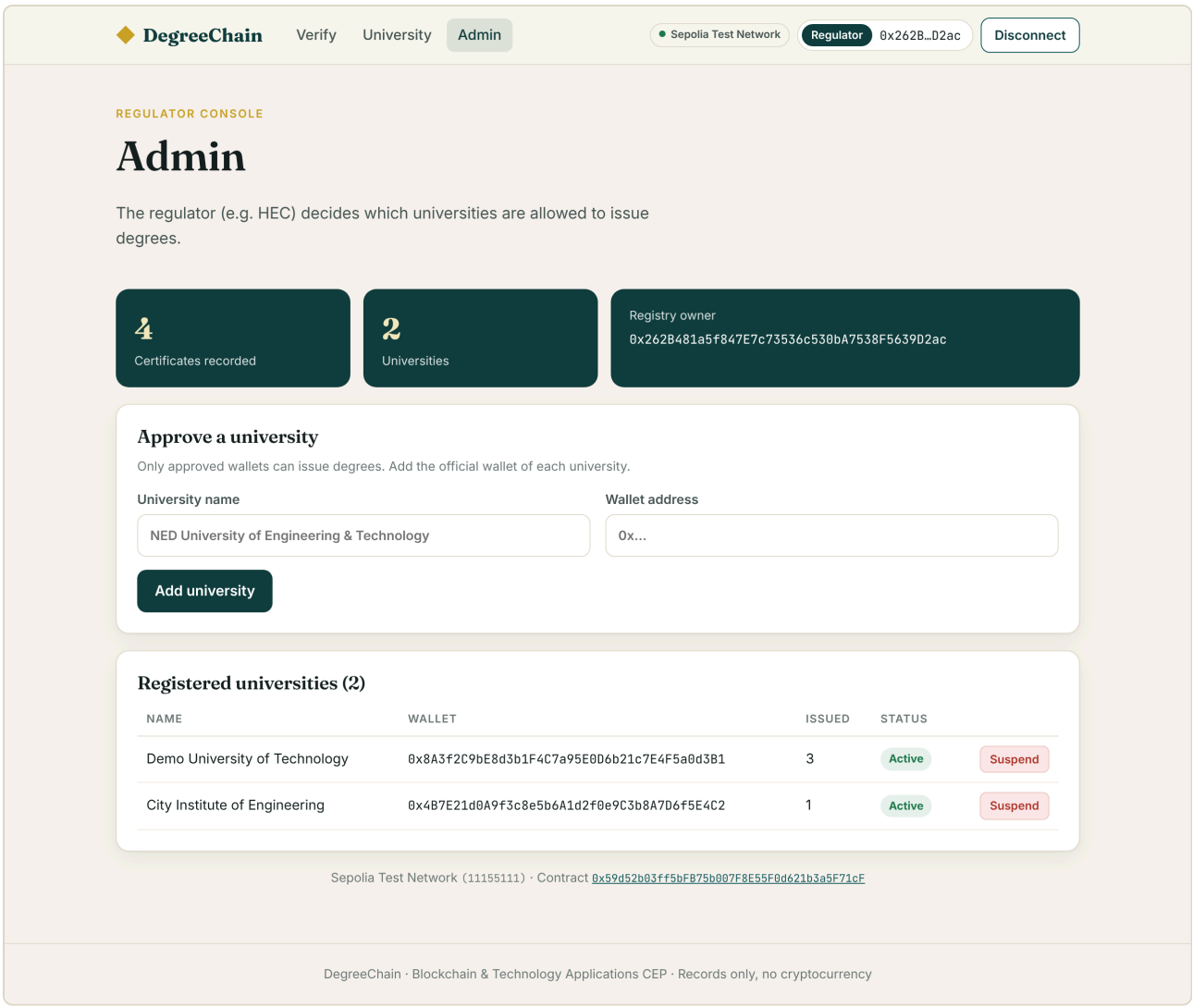


Figure 9. Admin console: the regulator wallet is recognised; universities can be approved or suspended

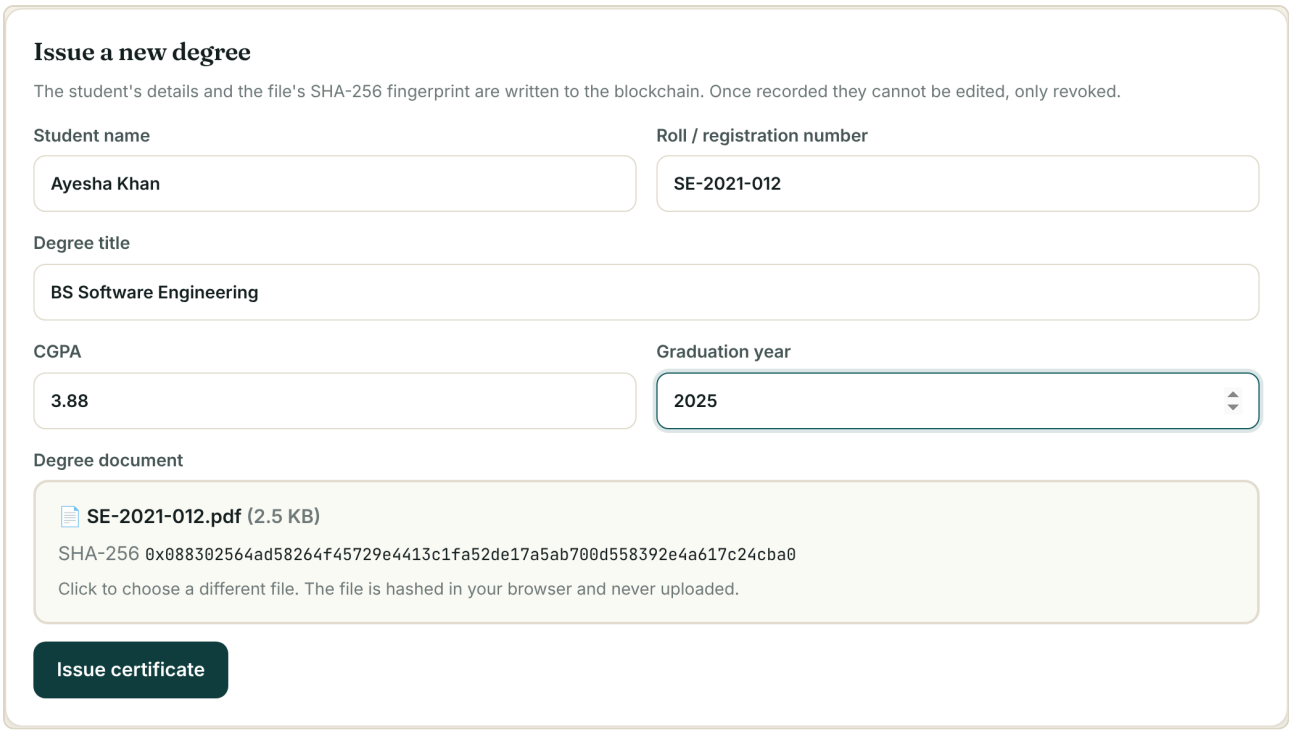


Figure 10. University portal: issuing a degree. The SHA-256 fingerprint of the attached PDF is computed in the browser

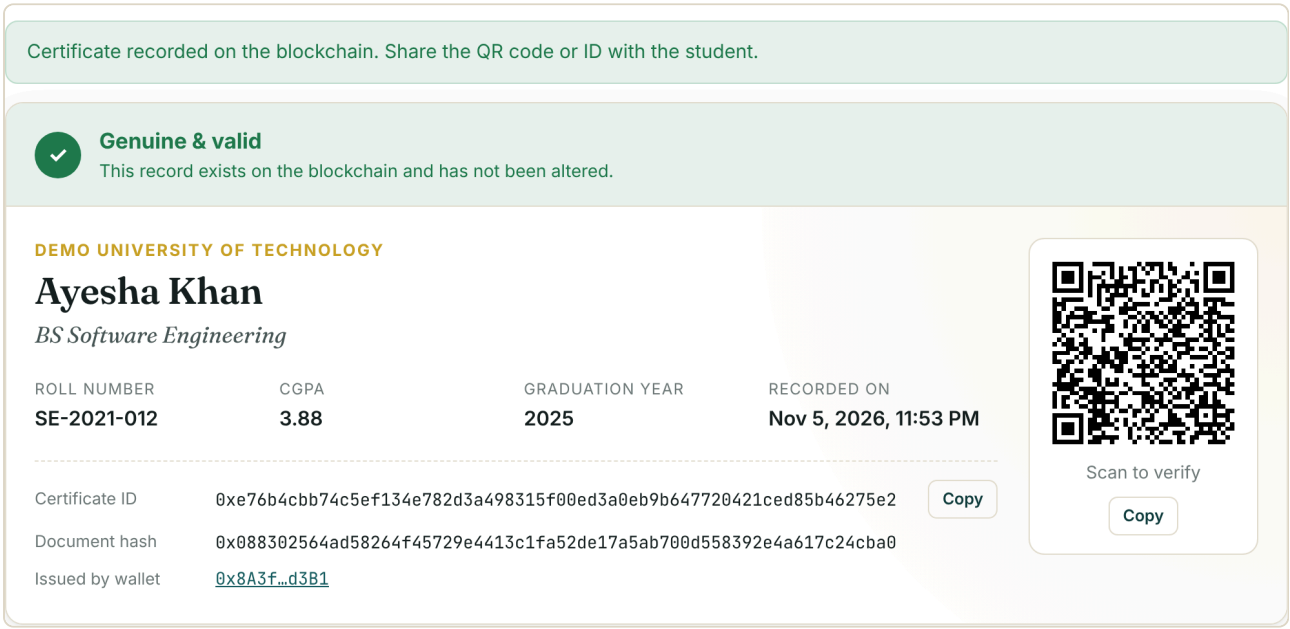


Figure 11. Certificate recorded on the blockchain with its permanent ID, document hash and QR code

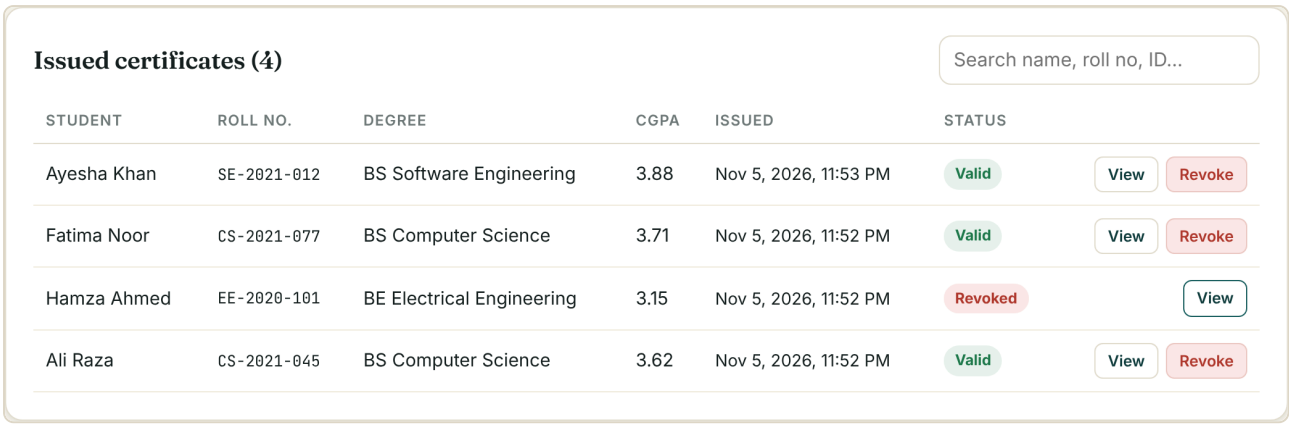


Figure 12. List of degrees issued by the university, with one revoked certificate

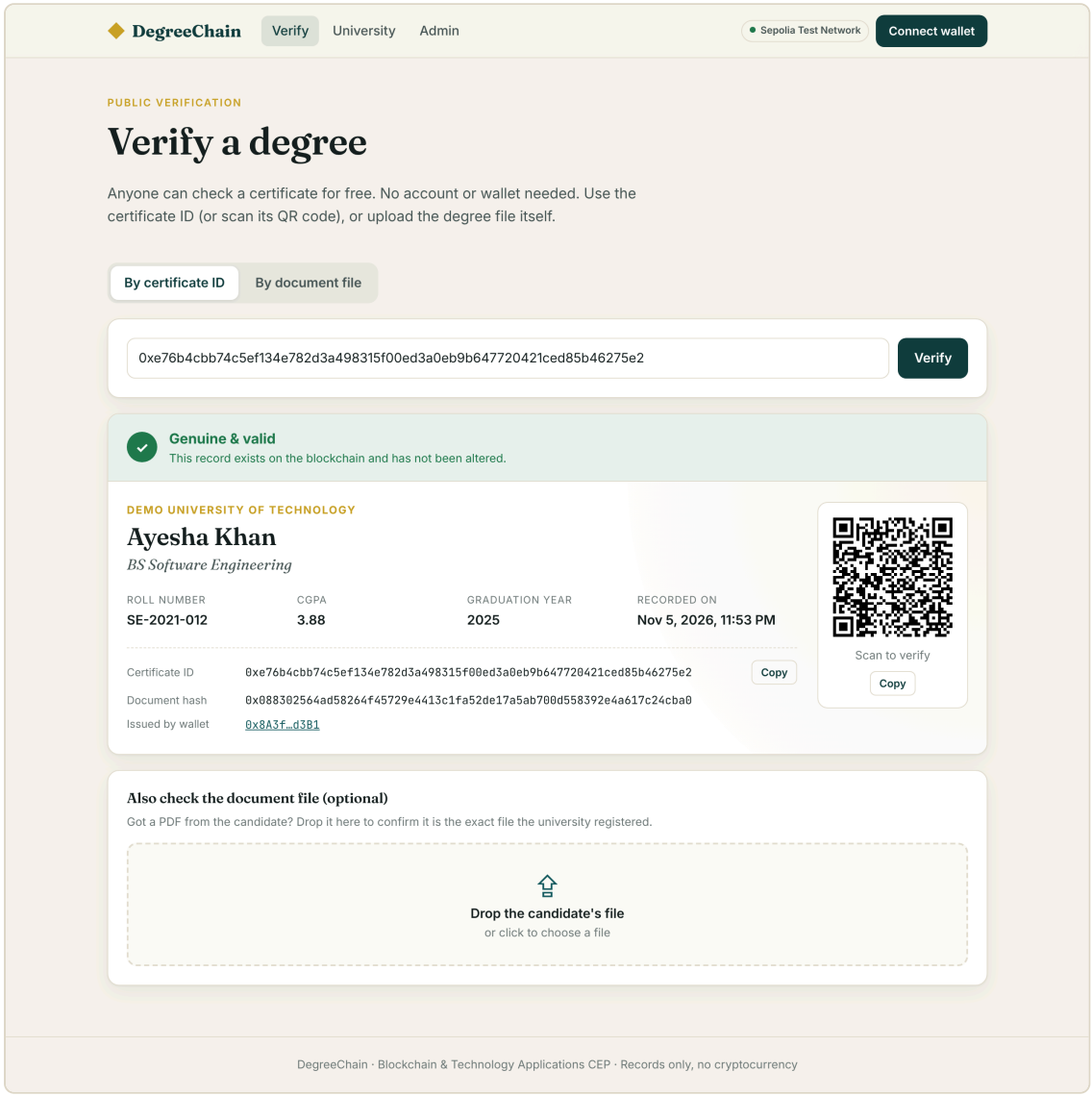


Figure 13. Public verification by certificate ID: Genuine and valid (no wallet needed)

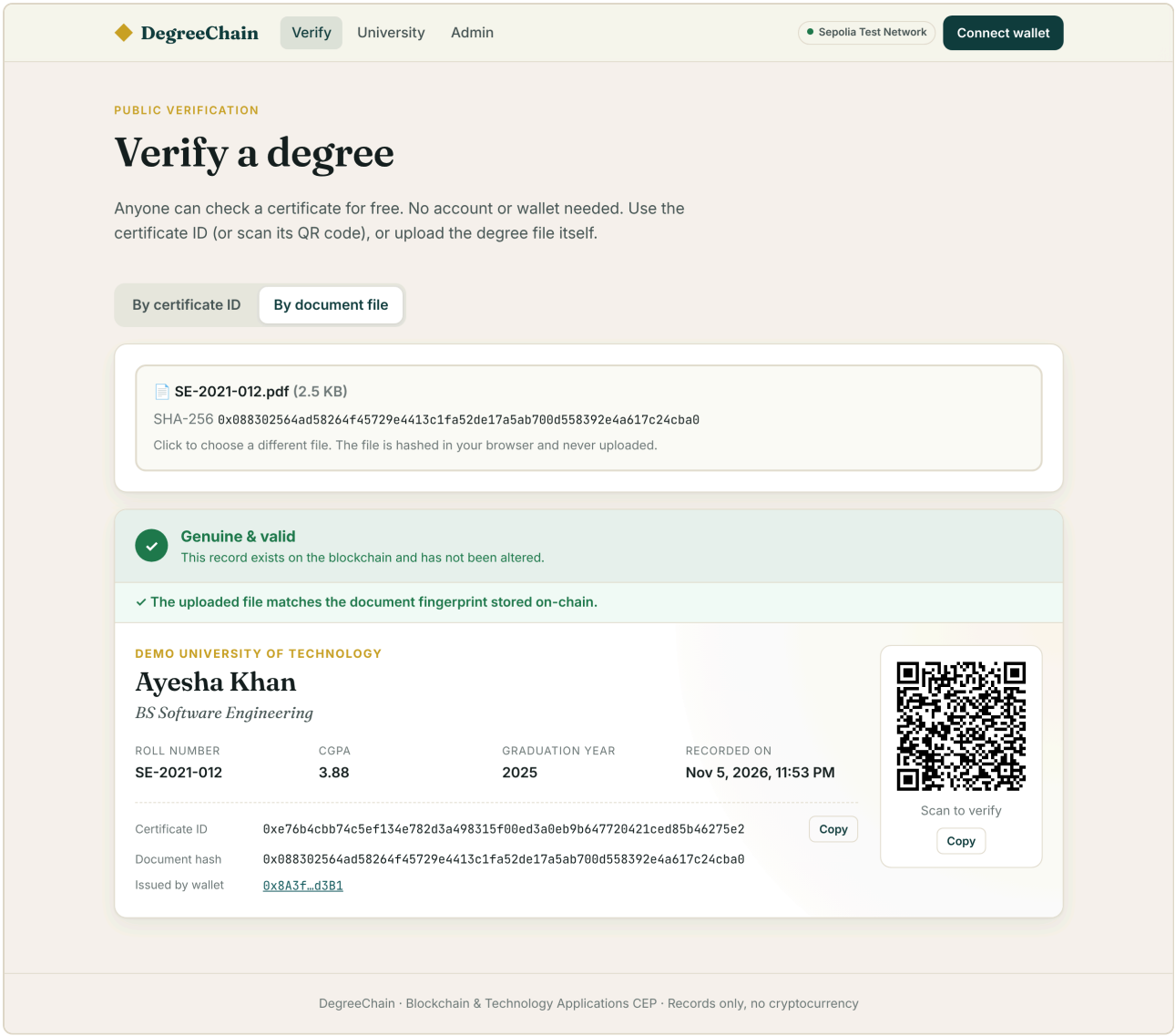


Figure 14. Verification by uploading the genuine PDF: the fingerprint matches the on-chain record

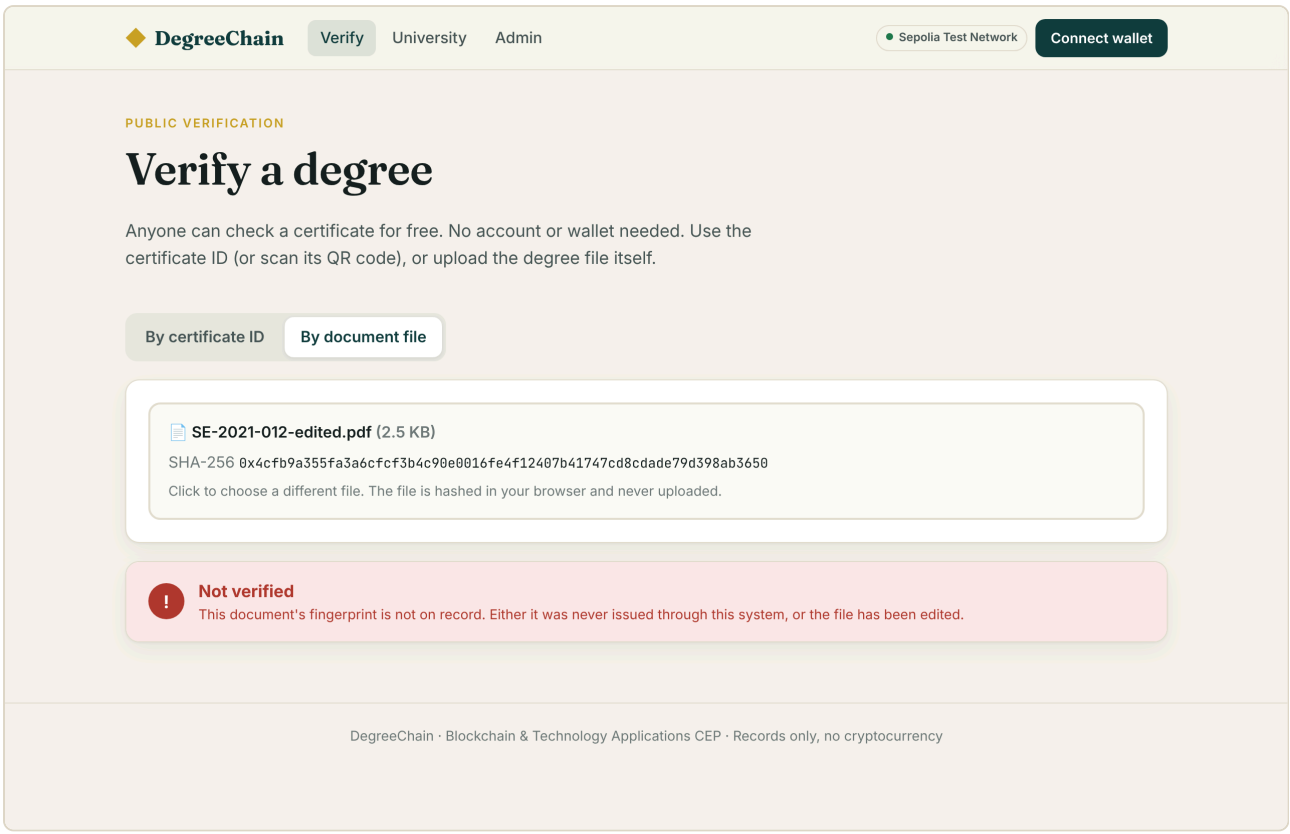


Figure 15. The same PDF with the CGPA edited from 3.88 to 3.98 is rejected: its fingerprint is not on record

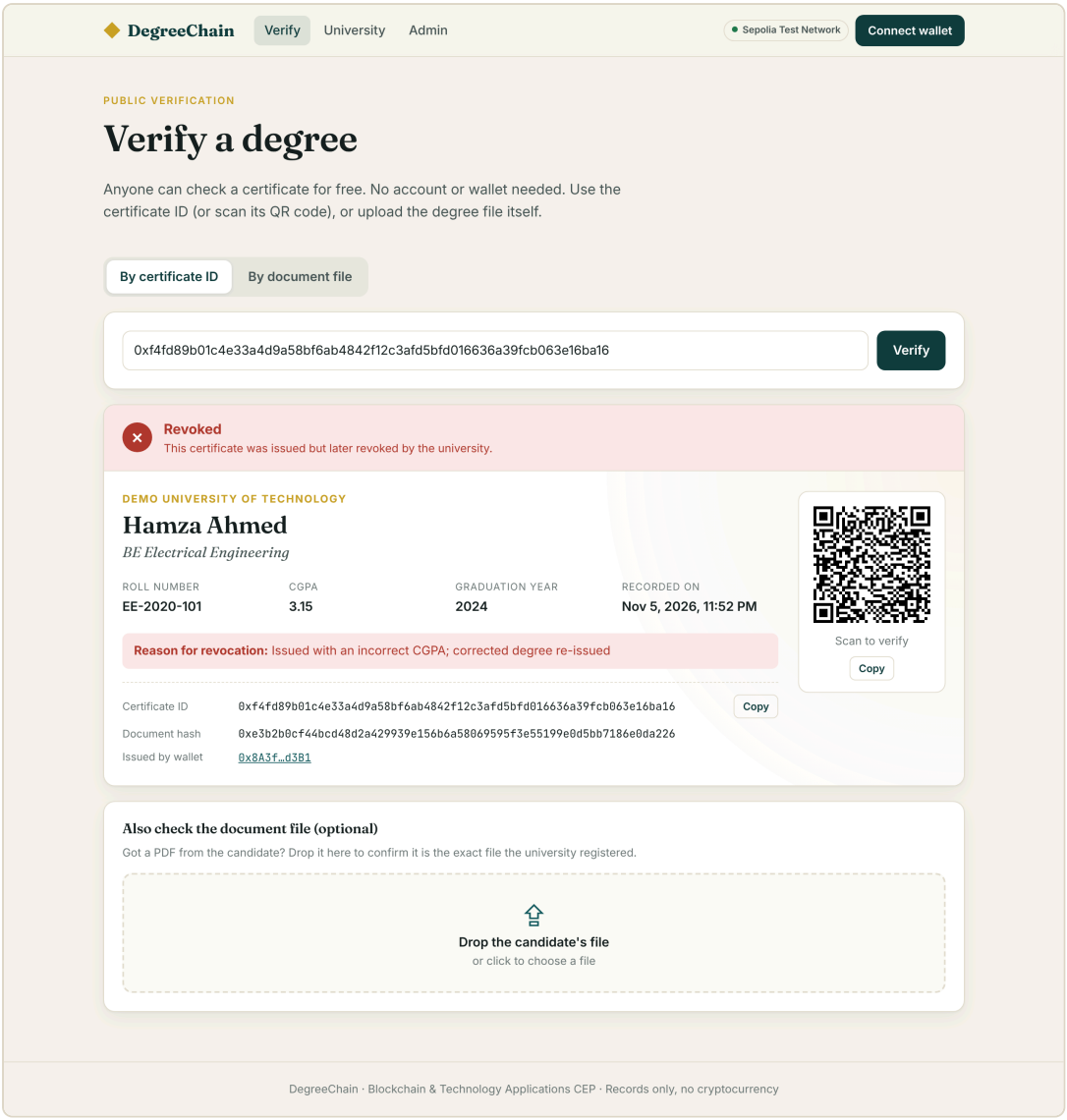


Figure 16. A revoked certificate is still visible, clearly marked, with the reason given by the university

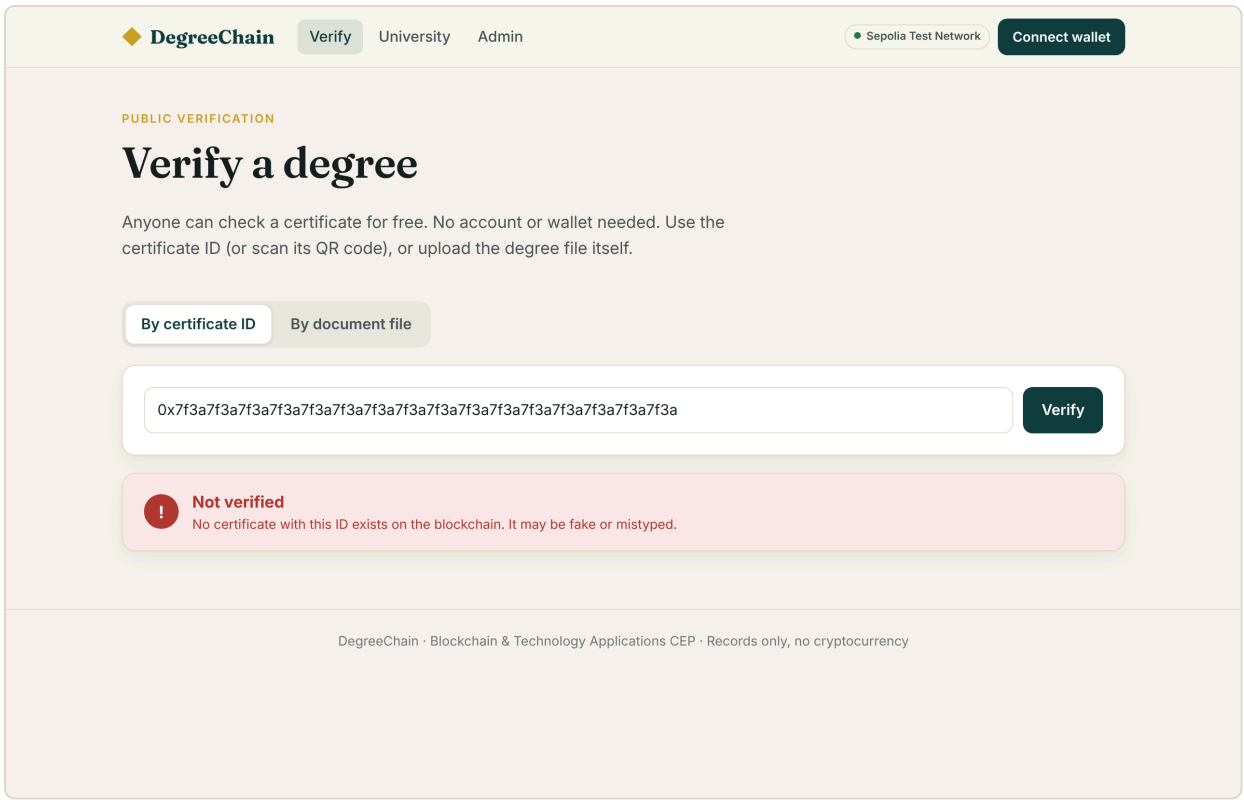


Figure 17. A certificate ID that was never issued is reported as not verified

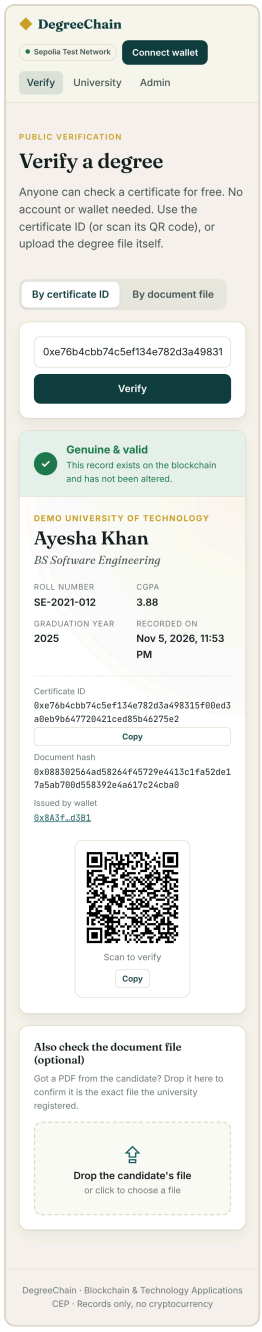


Figure 18. Verification result on a mobile phone

Responsive design

Employers will often scan the QR code with a phone, so the verification page adapts to small screens: the status banner stays at the top, the details collapse into two columns, and the QR code moves below the details.



Figure 19. Live statistics read from the contract on the Admin page

11. Testing and Results

11.1 Test approach

Testing was done at three levels:

1. **Unit tests** of the smart contract with Hardhat and Chai (19 test cases in `test/CertificateRegistry.test.js`), covering roles, issuing, verification and revocation.
2. **Contract rule checks** run directly against the compiled bytecode on a local EVM node (Foundry Anvil, configured with the Sepolia chain ID), sending real transactions and checking that each forbidden action is rejected with the correct custom error.
3. **End-to-end tests** of the production web build driven by an automated browser (Playwright), covering every page and outcome, plus a manual run of the full flow on the live Sepolia deployment.

11.2 Unit test cases

Group	Test cases
Deployment and roles (5)	Deployer is owner; owner registers universities; non-owners blocked; duplicate registration rejected; ownership transfer.
Issuing (8)	Issue emits event; every field stored correctly; only active universities issue; suspended university blocked; old certificates stay valid after suspension; duplicate document rejected; empty fields and empty hash rejected; certificates listed per university.
Verification (3)	Find by document hash; tampered or unknown document returns zero; unknown ID reported as not found.
Revocation (3)	Issuer can revoke with reason; other universities and outsiders blocked; double revoke, unknown ID and empty reason rejected.

11.3 Contract rule checks (executed)

#	Scenario	Expected result	Result
1	Outsider tries to approve a university	Rejected: `NotOwner`	**PASS**
2	Unapproved wallet tries to issue	Rejected: `NotActiveUniversity`	**PASS**
3	Issue with empty student name	Rejected: `EmptyField`	**PASS**
4	Issue with zero document hash	Rejected: `InvalidDocumentHash`	**PASS**
5	Another university registers an already-registered document	Rejected: `DocumentAlreadyRegistered`	**PASS**
6	Another university tries to revoke a certificate	Rejected: `NotIssuer`	**PASS**

#	Scenario	Expected result	Result
7	Revoke an already-revoked certificate	Rejected: `AlreadyRevoked`	**PASS**
8	Revoke an ID that does not exist	Rejected: `CertificateNotFound`	**PASS**
9	Suspended university tries to issue	Rejected: `NotActiveUniversity`	**PASS**
10	Certificates of a suspended university are verified	Still valid	**PASS**
11	Re-activated university issues again	Succeeds	**PASS**
12	Look up the hash of an edited PDF	No match (zero ID)	**PASS**
13	Look up the hash of the genuine PDF	Returns its certificate ID	**PASS**

11.4 End-to-end scenarios (executed)

#	Scenario	Observed result	Result
1	Owner wallet opens Admin page	Recognised as Regulator; 2 universities listed	**PASS**
2	University issues a degree with PDF	Certificate recorded; ID and QR shown	**PASS**
3	University revokes a degree with a reason	Status changes to Revoked in the list	**PASS**
4	Verify by ID (QR link)	Genuine and valid	**PASS**
5	Verify by uploading genuine PDF	Genuine; file matches fingerprint	**PASS**
6	Verify edited PDF (CGPA changed)	Not verified	**PASS**
7	Verify revoked certificate	Revoked, with reason	**PASS**
8	Verify an ID that was never issued	Not verified	**PASS**
9	Mobile screen (390 px wide)	Layout adapts, QR centred	**PASS**
10	Full flow on live site with Sepolia contract	Approve, issue, verify and tamper check all worked	**PASS**

12. Security, Privacy and Cost Analysis

12.1 Threats and mitigations

Threat	Mitigation
Forged certificate made from scratch	It has no certificate ID on-chain and its file hash is not registered, so it fails verification.
Genuine certificate edited (grade, name, year)	Any change alters the SHA-256 hash; file verification fails (Figure 15).
Fake "university" issues degrees	Only wallets approved by the regulator can issue (`onlyActiveUniversity`).
Insider changes a stored record	The contract has no update function; on-chain data cannot be modified.
A university revokes another's degree	Only the original issuer can revoke (`NotIssuer`).
Compromised university wallet	Regulator suspends the wallet immediately; damaged certificates can be revoked by the issuer.
Replay of a document as a new degree	Each document hash can be registered only once.
Private key exposure during deployment	The contract was deployed from the browser via MetaMask; the key never left the wallet.

12.2 Privacy

Degree files are never uploaded: hashing happens in the browser. The on-chain record does contain the student's name, roll number, degree title and CGPA in readable form, which is necessary for a verifier to see what was issued but means this information is public. Section 13 discusses how a production system could reduce this.

12.3 Transaction cost (gas)

Gas was measured on the local EVM node using the same compiled bytecode. Verification costs nothing because it is a read-only call.

Operation	Who pays	Gas used (measured)
Deploy contract (one time)	Regulator	≈ 1,327,000
Approve a university	Regulator	≈ 100,000 to 117,000
Issue a certificate	University	≈ 287,000 to 321,000 (depends on text length)
Revoke a certificate	University	≈ 118,000
Verify (by ID or by file)	Nobody	0 (free view call)

On Sepolia these fees are paid in free test ETH. The whole demonstration (deployment, approvals and several certificates) used only a small fraction of the 0.05 test ETH obtained from a faucet.

13. Limitations and Future Work

13.1 Limitations

- **Personal data on a public chain.** Names, roll numbers and CGPA are readable by anyone and cannot be deleted.
- **Test network.** Sepolia is for testing; a real deployment would need a production network and a plan for transaction fees.
- **Single regulator key.** One owner wallet controls university approvals; if lost, ownership cannot be recovered.
- **Trust in the initial record.** The blockchain proves a record was not changed after issue, not that the university entered correct data.
- **Wallet requirement for institutions.** University staff must use MetaMask, which may be unfamiliar.

13.2 Future work

- Store only a hash of the personal details on-chain (or use a permissioned chain) and reveal details only to authorised verifiers.
- Multi-signature regulator wallet so that approvals need several officials.
- Bulk issuing from a CSV file for an entire graduating batch.
- Email or SMS delivery of the certificate ID and QR code to students.
- Generate the degree PDF inside the system with the QR code printed on it.
- Integration with the HEC Degree Attestation System and university ERPs.

14. Conclusion

DegreeChain shows that a blockchain can solve a concrete, local problem, the fake and unverifiable degree, without any cryptocurrency features. By storing the details and SHA-256 fingerprint of each degree in a smart contract, the system makes degree records permanent and tamper-evident, lets a regulator control who can issue, keeps an auditable trail of revocations, and turns a verification process that takes days into a free check that takes seconds and needs no account. The project was implemented end to end: a tested Solidity contract deployed on the Ethereum Sepolia network and a responsive React application publicly available at `degreechain.vercel.app`. All executed tests passed, including the key case where a degree file edited by a single grade point was correctly rejected.

References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
2. G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger" (Ethereum Yellow Paper).
3. Solidity Documentation, version 0.8.24. https://docs.soliditylang.org
4. National Institute of Standards and Technology, FIPS 180-4: Secure Hash Standard (SHS), 2015.
5. ethers.js v6 Documentation. https://docs.ethers.org
6. EIP-1193: Ethereum Provider JavaScript API. https://eips.ethereum.org/EIPS/eip-1193
7. Blockcerts: The Open Standard for Blockchain Credentials. https://www.blockcerts.org
8. Hardhat Documentation. https://hardhat.org/docs
9. MDN Web Docs, SubtleCrypto.digest(). https://developer.mozilla.org
10. D. Walsh, "Fake Diplomas, Real Cash: Pakistani Company Axact Reaps Millions," *The New York Times*, May 17, 2015.
11. Higher Education Commission of Pakistan, Degree Attestation System. https://hec.gov.pk

Appendix A: Smart Contract API

Function	Access	Purpose
`addUniversity(address, name)`	Owner	Approve a university wallet
`setUniversityStatus(address, active)`	Owner	Suspend or re-activate (old certificates stay valid)
`transferOwnership(newOwner)`	Owner	Hand the registry to a new regulator wallet
`issueCertificate(name, roll, degree, cgpa, year, docHash)`	Active university	Record a degree; returns the certificate ID
`revokeCertificate(certId, reason)`	Issuing university	Revoke with a reason
`verifyCertificate(certId)`	Anyone (free)	Returns exists, valid, the record and university name
`findByDocumentHash(hash)`	Anyone (free)	Certificate ID for a document, or zero
`getUniversities()`, `getUniversity(addr)`	Anyone (free)	University list and details
`getCertificatesByUniversity(addr)`	Anyone (free)	IDs issued by a university

Events

`OwnershipTransferred`, `UniversityAdded`, `UniversityStatusChanged`, `CertificateIssued(certId, issuer, documentHash, rollNumber)`, `CertificateRevoked(certId, issuer, reason)`.

Custom errors

`NotOwner`, `NotActiveUniversity`, `InvalidAddress`, `EmptyField`, `UniversityAlreadyRegistered`, `UniversityNotRegistered`, `InvalidDocumentHash`, `DocumentAlreadyRegistered`, `CertificateNotFound`, `NotIssuer`, `AlreadyRevoked`.

Appendix B: Running the Project

Use the live system

1. Open `https://degreechain.vercel.app`. Verification works without any wallet.
2. For Admin and University pages, install MetaMask, select the Sepolia network and connect the approved wallet.

Run locally

```
git clone https://github.com/shahid786-dev/Blockchain-Based-Degree-Verification
cd Blockchain-Based-Degree-Verification
npm install && npm run frontend:install
npm test                 # run the 19 contract unit tests
npm run node             # terminal 1: local blockchain
npm run deploy:local     # terminal 2: deploy + register Demo University
npm run seed:local       # optional: 3 sample certificates
npm run frontend:dev     # open http://localhost:5173
```

Repository structure

```
contracts/CertificateRegistry.sol    # smart contract
scripts/deploy.js, seed.js           # deployment and sample data
test/CertificateRegistry.test.js     # 19 unit tests
frontend/src/contract/               # ABI, bytecode, deployments/sepolia.json
frontend/src/lib/                    # blockchain helpers, SHA-256 hashing
frontend/src/pages/                  # Home, Verify, University, Admin, Deploy
frontend/src/components/             # CertificateCard, FileDrop, Navbar, ...
```